Able Lubes API Guide

Product catalog, opening quantity, and stock adjustments

Base URL: /api/v1 **Content-Type:** application/json

All endpoints require a valid bearer token. Send **Authorization: Bearer <token>**. Responses use a success flag and data object; lists include pagination metadata.

Recommended setup flow

Step	Action
1	Create brands and units for product selection.
2	Create products with brandId, unitId, selling price, and optional qty. qty initializes currentStock.
3	Use purchases to add stock and sales to deduct stock.
4	Use stock adjustments for physical count corrections. Adjustments require an admin token and a reason.

Important behavior

Product PATCH cannot change stock. Use POST /stock-adjustments to change stock after creation. Adjustment and movement records are written atomically; OUT adjustments that would make stock negative are rejected. SKU is removed from product creation, updates, search, and responses.

Migration compatibility

Existing products retain their stock. Existing text units are converted into Unit records. Existing products are assigned the placeholder brand “Unassigned” until updated. Existing SKU values are dropped by the migration.

Catalog endpoints

Brand and Unit support paginated listing, create, read by ID, and PATCH updates. Deactivate with active:false. Products cannot be created or assigned to inactive brands or units.

Method and path	Purpose / request
GET /brands?page=1&limit;=20&search;=Mobil	List brands; search by name.
POST /brands	Create: {"name":"Mobil","active":true}
GET /brands/{id}	Fetch one brand.
PATCH /brands/{id}	Update name or active state.
GET /units?page=1&limit;=20	List selectable units.
POST /units	Create: {"name":"Litre","symbol":"L","active":true}
GET /units/{id}	Fetch one unit.
PATCH /units/{id}	Update name, symbol, or active state.

Create a product

```
POST /products
{
"name": "Engine Oil 5W-30",
"brandId": "<brand-uuid>",
"unitId": "<unit-uuid>",
"sellingPrice": 850,
"mrp": 900,
"qty": 12
}
```

qty is optional, non-negative, and defaults to 0. The response includes currentStock as a decimal string plus related brand and unit. A positive opening quantity creates an OPENING_STOCK movement associated with the signed-in user.

Product endpoints

Method and path	Purpose
GET /products?search=oil&active;=true	Paginated product list; searches product and brand names.
GET /products/{id}	Get product, brand, unit, and current stock.
PATCH /products/{id}	Update name, brandId, unitId, sellingPrice, mrp, or active. No qty/currentStock.

Stock adjustment endpoints

Method and path	Purpose
POST /stock-adjustments	Create an admin-only manual stock adjustment.
GET /stock-adjustments?page=1&limit;=20	List movement history, newest first.
GET /stock-adjustments?productId=	Filter history by product.

Create adjustment

```
POST /stock-adjustments
{
"productId": "<product-uuid>",
"quantity": 3,
"direction": "OUT",
"reason": "Physical count correction",
"notes": "Counted on shelf"
}
```

Use direction IN to add stock or OUT to remove stock. quantity must be positive and supports three decimal places. The response contains movement details, stockAfter, product, and creating user.

Errors to handle

HTTP	Meaning
400	Invalid fields or missing/inactive brand or unit.
401	Missing, invalid, or expired bearer token.
403	Insufficient permission; stock adjustment is ADMIN-only.
404	Product, brand, or unit is unavailable.
409	OUT exceeds available stock or a concurrent stock update occurred.

Example success response (shape)

```
{
"success": true,
"data": {
"id": "<movement-uuid>",
"productId": "<product-uuid>",
"type": "ADJUSTMENT",
"direction": "OUT",
"quantity": "3",
"stockAfter": "9",
"reason": "Physical count correction",
"createdAt": "<ISO-8601 timestamp>"
}
}
```

Interactive API docs: /docs when the backend is running.

Coupon API

Coupon amounts are fixed currency discounts. Codes are trimmed and stored in uppercase. Each code is unique, and each sale can have at most one sale-based coupon.

Coupon record endpoints

Method and path	Behavior
GET /coupons?type=all	List all coupons (default).
GET /coupons?type=sales-based	Only coupons applied to a sale.
GET /coupons?type=non-sale-based	Only standalone records.
GET /coupons/{id}	Fetch one coupon.
POST /coupons	Create standalone coupon: {"code":"SAVE100","price":100}. Does not change any sale.
PATCH /coupons/{id}	Update a standalone coupon code or price.
DELETE /coupons/{id}	Delete a standalone coupon. Sale-based coupons cannot be edited or deleted here.

Create a sale with coupon

```
POST /sales
{
"customerId": "<customer-uuid>",
"items": [{ "productId": "<product-uuid>", "quantity": 1 }],
"couponCode": "SAVE100",
"couponPrice": 100
}
```

Provide couponCode and couponPrice together. The backend subtracts couponPrice from the invoice total and creates a sale-based coupon record linked to the new sale, in the same transaction. If the coupon discount exceeds the remaining sale value or the code already exists, the request is rejected.

Apply or replace coupon on an unpaid sale

```
PATCH /sales/<sale-uuid>
{ "couponCode": "SAVE150", "couponPrice": 150 }
```

PATCH applies a coupon to an existing posted sale or replaces its existing coupon. The sale must have no recorded payments. Only the coupon fields are editable through this PATCH route; line items and other sale fields are not changed.

Coupon response includes code, price, isSaleBased, saleId, createdAt, and updatedAt. List results are paginated with page and limit.